PrepLang

A Data Aware Compiler for Table and Image Preprocessing

Compiler Design Laboratory (BCSE307P)

Phase 2: Core Implementation

V.Nikhil

24BAI0073

1. Project Summary

PrepLang is a small language I designed for cleaning data before it is used for machine learning. It works with CSV and Parquet tables, whole folders of tables, and folders of images. I wrote the compiler for it in C++17. The compiler reads a PrepLang program, checks it, and then writes a Python program that does the real cleaning, using pandas for tables and OpenCV for images.

The main idea that makes it different is simple: the compiler opens the actual data files while it is compiling. Because of that it already knows the type of every column, whether any values are missing, the smallest and largest values, and the size and channels of every image. So if a program uses a column that does not exist, puts text into a number column, tries to process a broken image, or saves decimal pixels as a PNG file, the compiler points out the mistake straight away. The user does not have to wait for a script to crash halfway through the data.

Here is a small example program:

```
LOAD "students.csv" AS students;
EXPECT students.marks BETWEEN 0 AND 100;
FILL_NULL students.department WITH "UNKNOWN";
DROP_NULL students;
NORMALIZE students.attendance USING MINMAX;
FILTER students WHERE marks >= 50 AND attendance > 0.5;
SELECT students [name, age, marks];
SAVE students AS "clean.csv";
```

2. Progress in Phase 2

Every module I planned in Phase 1 is now built and working. Here is where each one lives in the source code:

- **Lexical analysis** in src/lexer
- **Syntax analysis** in src/parser
- **Abstract syntax tree** in src/ast
- **Symbol table** in src/sema/SymbolTable.cpp
- **Semantic analysis** in src/sema
- **Reading the data files** in src/schema
- **Intermediate code** in src/ir
- **Code optimization** in src/optimizer
- **Target code generation** in src/backend
- **Error handling** in src/common/Diagnostics.cpp
- **Testing** with 141 unit tests in 7 test programs, and all of them pass

In total the project has about 7,500 lines of C++, 8 example programs and 5 programs that show the error messages. The full source code is on GitHub at github.com/Nikhi00718/preLang.

3. System Architecture

The compiler works as a chain of phases, where each phase hands its result to the next one. If any phase finds a mistake, the compiler stops there and prints the errors, so the same mistake is not reported again by the later phases.

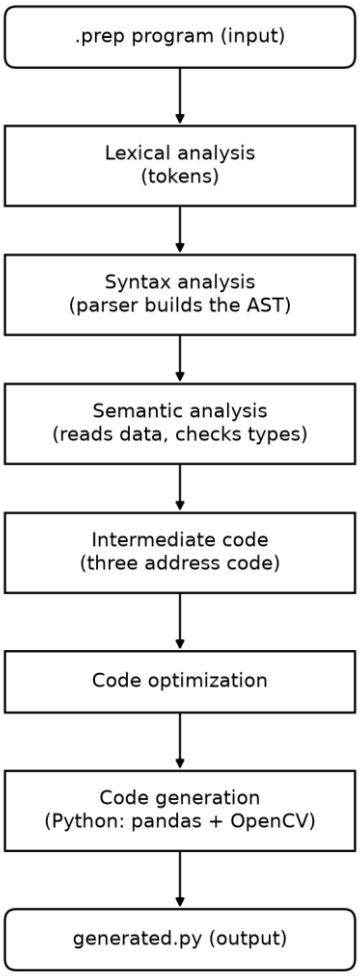


Figure 1: Phases of the PrepLang compiler

4. How Each Module Works

4.1 Lexical Analysis

The lexer reads the program one character at a time and groups the characters into tokens such as keywords (LOAD, FILTER and so on), names, numbers, strings, operators and punctuation. Keywords can be written in any case, and anything after a # is treated as a comment. Every token remembers its line and column so that error messages can point to the exact place. If the lexer finds a character it does not know, like $, or a string that is never closed, it reports a lexical error.

4.2 Syntax Analysis

I wrote the parser by hand using recursive descent. Since every statement starts with its own keyword, looking at one token is enough to decide which rule to use, so the grammar is LL(1). Conditions inside FILTER follow the usual order of operators: OR is weakest, then AND, then NOT, and comparisons bind the tightest.

When the parser meets a syntax error it does not give up. It skips ahead to the next semicolon and carries on, so one run can report several errors. If a keyword is misspelled, it suggests the closest correct one. For example, writing NORMALISE gives the hint “did you mean NORMALIZE?”.

4.3 Abstract Syntax Tree

While parsing, the parser builds an abstract syntax tree with one kind of node for each statement and each condition. The later phases walk over this tree using the visitor pattern. The tree can be printed in the terminal, or saved as a Graphviz picture.

4.4 Symbol Table

The symbol table keeps track of every dataset in the program: its name, what kind of data it is (a table, a folder of tables, an image, a folder of images, or images linked with labels) and the line where it was created. For every column it also stores the type, whether values can be missing, and the range of values. For images it stores the size, the number of channels and the pixel type.

The table is updated after every statement. For example, after FILTER marks >= 50 the compiler knows that marks now lies between 50 and 95.

4.5 Semantic Analysis

This is the heart of the project. Using the symbol table and the real data, the semantic analyzer checks that every operation makes sense. It reports a type error if you try to normalize a text column, and it catches names that were never defined, datasets defined twice, columns that do not exist, and columns used after they were dropped.

It also decides every EXPECT contract. A contract is marked proven when the known facts show it is always true, violated when the data clearly breaks it, and otherwise it is left to be checked when the program runs. For images it catches broken files, decimal pixels being saved as PNG or JPG, and transparent images being saved as JPG. Figures 2 and 3 show how a statement and a contract are checked.

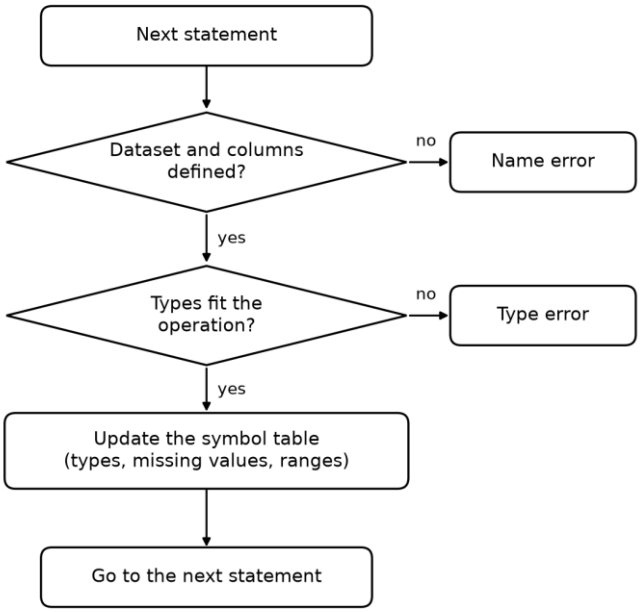


Figure 2: How semantic analysis checks one statement

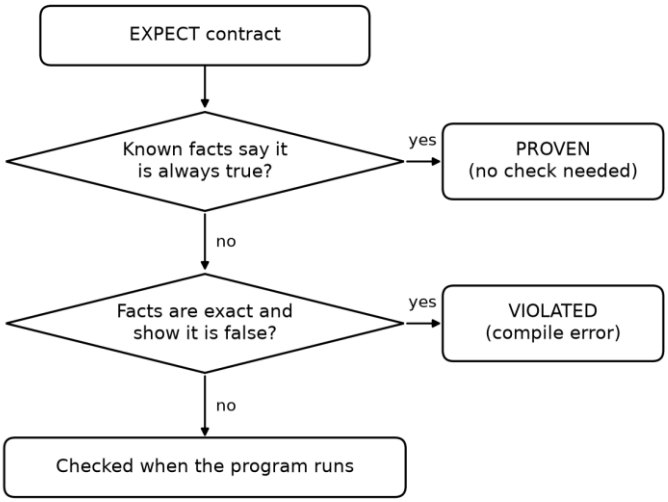


Figure 3: How an EXPECT contract is decided

4.6 Reading the Data Files

For a CSV file, the compiler reads every cell once to learn each column's type, its missing values and its smallest and largest value. For a Parquet file it only reads the small summary stored at the end of the file, so even a file with 16 million rows is analysed in about 0.02 seconds. For images it only reads the header of each file (the IHDR part of a PNG or the SOF part of a JPEG), so it never has to load the pixels.

4.7 Intermediate Code

After checking, the compiler turns the tree into three address code, stored as quadruples. A wildcard such as company.*.age is expanded into one instruction for every table. Conditions

in FILTER are broken into temporaries, for example t1 = marks >= 50, then t2 = attendance > 0.5, then t3 = t1 AND t2.

4.8 Code Optimization

The optimizer removes work that is not needed. It deletes operations that would do nothing, such as filling missing values in a column that has none. It removes run time checks for contracts that were already proven. It removes temporaries and results that are never used, and it makes the program read only the columns that are actually needed. For images, it joins all the steps into one loop that handles one image at a time, which keeps memory use low.

4.9 Target Code Generation

Finally the compiler writes a single Python file, with pandas code for tables and OpenCV code for images. The generated program first checks a CRC32 fingerprint of every input file, and it refuses to run if a file has changed since it was compiled. This keeps the proofs made by the compiler valid.

4.10 Error Handling

Every error message says which phase found it, the line and column, draws a caret (^) under the exact spot, and gives a hint on how to fix it. Errors are sorted by line, and all errors from one phase are shown together.

5. Intermediate Results

These are real outputs from each phase when the compiler runs on examples/tabular/students.prep.

5.1 Tokens

```
LINE:COL  TYPE             CATEGORY     LEXEME
2:1       LOAD             KEYWORD      LOAD
2:6       STRING_LITERAL   LITERAL      "students.csv"
2:21      AS               KEYWORD      AS
2:24      IDENTIFIER       IDENTIFIER   students
2:32      SEMICOLON        PUNCTUATION  ;
```

5.2 Syntax Tree

```
Program
|   Load TABLE
|   |   file: "students.csv"
|   |   dataset: students
|   Expect
|   |   target: students.marks
|   |   contract: BETWEEN 0 AND 100
```

5.3 Semantic Analysis

```
line 2: TABLE students: students (8 rows, 6 columns)
```

```
line 5: EXPECT students.marks BETWEEN 0 AND 100: PROVEN, values are in [35,
95]
line 9: DROP_NULL removes rows of students (rows with missing age)
line 10: NORMALIZE students.attendance: FLOAT in [0, 1]
line 12: FILTER students: marks is INTEGER [50, 95]
line 16: EXPECT students.marks BETWEEN 60 AND 100: checked at run time
```

5.4 Optimization Report (for examples/tabular/optimize.prep)

```
contract check elimination: removed CHECK top.marks BETWEEN 0 AND 100
no effect elimination: removed FILL_NULL top.marks WITH 0
no effect elimination: removed FILTER top IF t1
dead code elimination: removed t1 = top.marks >= 0
column pruning: LOAD top reads 2 of 6 columns
dead store elimination: removed NORMALIZE top.attendance
Instructions: 14 before, 9 after
```

6. Testing

6.1 Unit Tests

I wrote 141 unit tests, split into seven test programs, and all of them pass when run with ctest:

- **test_lexer** has 15 tests for tokens, keywords, numbers, strings and lexical errors.
- **test_parser** has 29 tests for every statement, operator order, error recovery and suggestions.
- **test_semantic** has 29 tests for types, names, missing values, ranges and contracts.
- **test_tableset** has 13 tests for folders of tables, wildcards and contracts between tables.
- **test_imageset** has 20 tests for image headers, sizes, channels and image errors.
- **test_parquet** has 22 tests for Parquet files, their statistics and contracts.
- **test_optimizer** has 12 tests, one or more for each optimization.

6.2 Sample Test Cases

Below are some of the cases I tried by hand. For each one I wrote what I expected, and what the compiler actually printed. All of them behaved as expected.

1. Writing NORMALISE students.age should give a syntax error with a suggestion. The compiler replied: did you mean 'NORMALIZE'?
2. A stray $ in the program should give a lexical error. The compiler replied: unexpected character '$'.
3. NORMALIZE students.name should fail because name is text. The compiler replied: NORMALIZE needs a numeric column.
4. FILTER students WHERE marks > "excellent" compares a number with text. The compiler replied: cannot compare INTEGER column 'marks' with text.
5. NORMALIZE students.mark uses a column that does not exist. The compiler replied: column 'mark' does not exist, did you mean 'marks'?

6. Using a column after dropping it should be an error. The compiler replied: column 'age' was removed at line 12.
7. EXPECT students.marks BETWEEN 0 AND 100 should be proven, with no check left in the output. The compiler replied: PROVEN, values are in [35, 95].
8. A folder with one broken image should be caught before any processing. The compiler replied: 1 of 7 files cannot be decoded.
9. Saving decimal pixels as PNG should be an error instead of producing black images. The compiler replied: cannot save FLOAT32 pixels as PNG.
10. When the correct students.prep is compiled and run, it should write the cleaned file. The program printed: Saved 4 rows to clean.csv.

6.3 What an Error Message Looks Like

```
error[syntax]: expected a statement (LOAD, FILTER, RESIZE, SAVE, ...), found
'NORMALISE'
  --> examples/errors/invalid.prep:4:1
   |
4 | NORMALISE students.age;
   | ^
   = hint: did you mean 'NORMALIZE'?
```

7. Running the Demo

Figure 4 shows the steps I follow in the demo.

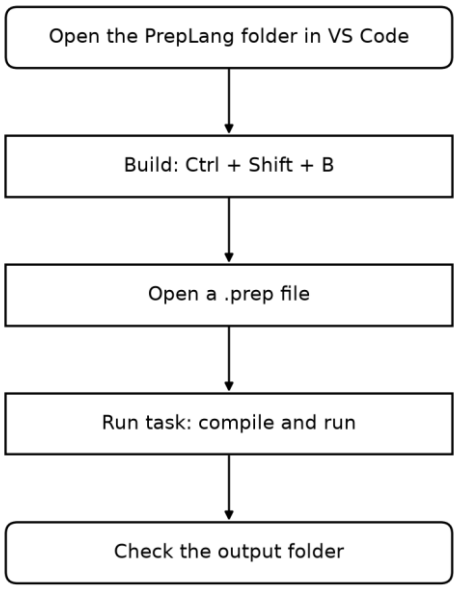


Figure 4: Steps to run the demo

11. Open the PrepLang folder in VS Code and press Ctrl + Shift + B to build the compiler.

12. Open examples/tabular/students.prep, then choose Terminal, Run Task, and “PrepLang: compile and run the open .prep file”. The output of every phase is printed, and the cleaned file appears in the output folder.
13. To show the error messages, open any file in examples/errors and run the same task.
14. To run the tests, choose Terminal, Run Task, and “PrepLang: run all tests”.

The same steps from the command line:

```
cmake -S . -B build -G "Visual Studio 17 2022"
cmake --build build
build\Debug\preplang.exe examples\tabular\students.prep --run
ctest --test-dir build -C Debug
```

8. Problems I Faced and How I Solved Them

- **Showing several syntax errors in one run.** I used panic mode recovery, where the parser skips to the next semicolon after an error and keeps going.
- **Numbers written in scientific notation, like 6.4e5, were being read as text.** I added support for exponents to the number reader and wrote a test for it.
- **Finding image sizes without loading the images.** I read only the header at the start of each PNG and JPEG file.
- **Removing proven checks safely.** The generated program checks a CRC32 fingerprint of every input file before it does anything.
- **Big files made compiling slow.** For Parquet files the compiler now reads only the summary stored at the end of the file.
- **The generated pandas code was slow.** Reading the data first and converting the types afterwards made it about six times faster.

9. What Is New in This Project

- The compiler reads the real data while compiling, so data mistakes are found before the program ever runs.
- Data contracts written with EXPECT are proven, rejected, or checked at run time, and checks that are already proven are removed.
- Tables and images are handled by one language and one analysis.
- The optimizations never change the output. The only one that changes pixels slightly is used only when the user asks for it.

10. Plan for Phase 3

- Test the compiler with larger and more complex inputs and measure its performance.
- Write the complete project report and prepare the presentation with screenshots.
- Write the user instructions and prepare the final demonstration.